

【RAG实战-第1天】 RAG流程详解与优化方案： 案例+代码+图解

RAG流程图解

- 1.1 整体流程
- 1.2 离线处理流程
- 1.3 在线查询处理流程

Query模块

- 2.1 Query模块
- 2.2 Query 重写 (Query Rewrite)
- 2.3 Query 扩写 (Query Expansion)
- 2.4 HYDE (Hypothetical Document Embeddings)

离线解析模块

- 3.1 核心问题
- 3.2 实际案例
- 3.3 优化策略
- 3.4 示例代码

检索召回模块

- 4.1 case分析
- 4.2 混合检索 (BM25 + 向量)
- 4.3 Embedding 模型微调
- 4.4 检索结果重排 (Rerank)
- 4.5: 评估指标与效果评估
- 4.6 召回总结

生成阶段优化

- 5.1 核心问题
- 5.2 实际案例 (Case)
- 优化策略

进阶优化方案 (下一期)

- 更多课程联系微信：1978201801

一句话理解RAG：

RAG就像是AI的"开卷考试"模式——不必死记硬背所有知识，而是在需要回答问题（用户query）时能翻阅自己的"参考书"（知识库）找到相关资料，然后用自己的语言组织回答。

难点：

- 1、**离线知识库**：怎么构建知识库（企业或个人的知识都是各种格式、各种模态（文字、图片、表格）混杂的）
- 2、**在线检索**：query怎么找到自己想要的知识，找准+找全
- 3、**在线生成**：拿到了知识怎么生成我们想要的内容

初期掌握目标：

- 1、掌握2阶段（检索、生成）3模块（query、index、生成）的**13种常用优化方式**以及**什么场景下使用、怎么优化**
- 2、能用llamaindex搭建一个自己的研报/论文阅读助手

我们带入一个实际的场景去思考如何优化

金融保险公司 RAG 问答系统

- 1、5000分不同格式的文档，包含ppt、pdf、txt、视频、扫描图片等各种类型和模态的文件
- 2、用户query较为复杂，有基于公司制度（报销流程等）、销售策略、产品信息、市场情况等各类问题，需要走不同的链路

接下来，我们按照如下思路进行说明**RAG 4个核心模块（query、解析入库、召回、生成）**优化方案

- 1、**核心问题**
- 2、**实际案例case**
- 3、**优化策略+手撕代码**
- 4、**为什么这么做能解决**

RAG流程图解



1.1 整体流程

检索增强生成(RAG)系统是一种结合外部知识库与大语言模型的智能架构。该系统通过检索相关文档来增强生成模型的回答能力，使其能够提供更准确、及时和可靠的信息。

1.2 离线处理流程

离线处理是系统的准备阶段，主要完成知识库的建立与向量化：

1. **知识文档库**：系统首先需要建立一个包含各类文档、知识和信息的库，这是RAG系统的知识基础
2. **文档向量化**：通过Embedding引擎将文本内容转换为高维向量，捕捉文本的语义信息
3. **向量数据库**：存储所有文档的向量表示，并建立高效的检索索引，为后续的实时查询做准备

离线流程只需执行一次或定期更新，不参与用户的实时交互过程。

1.3 在线查询处理流程

在线处理是系统的运行阶段，负责处理用户实时查询并生成回答：

1. 用户查询：接收用户输入的问题或指令
2. 查询向量化：将用户查询转换为与文档相同向量空间的表示
3. 语义检索：在向量数据库中寻找与查询语义最相似的文档片段【解释在notebook】
4. 大语言模型：将原始查询与检索到的相关文档结合，增强模型的上下文
5. 生成回答：模型基于检索回来的背景知识增强后的上下文生成最终回答返回给用户

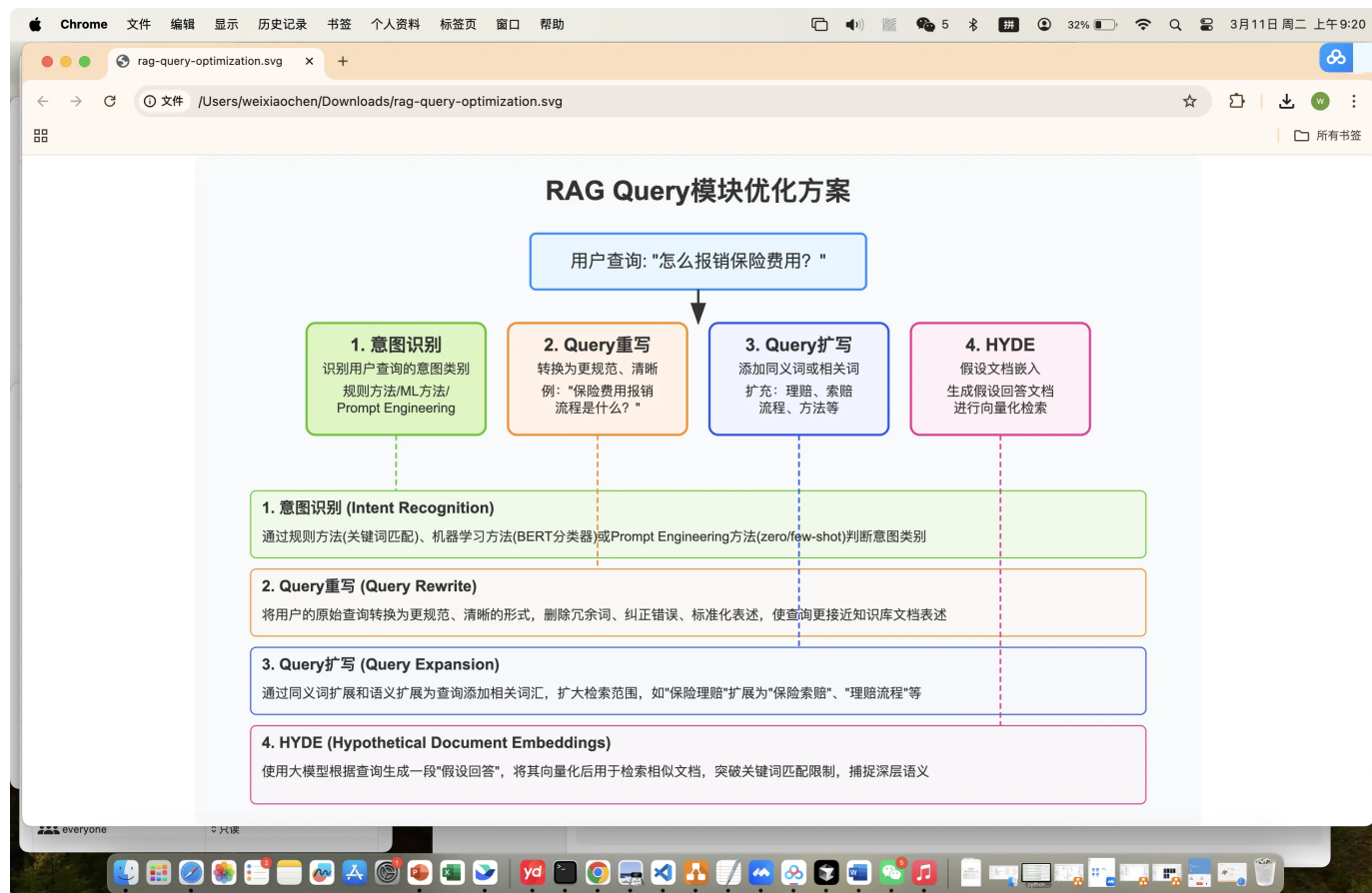
在线流程对每个用户查询都会实时执行，确保系统能够利用最新的知识库信息提供准确回答。

这两个流程相互配合，共同构成了RAG系统的完整工作机制，使大语言模型能够超越其训练数据的局限，提供**实时、准确、基于本地数据**的QA服务。

在实际工程中包含了非常多的策略优化，RAG核心优化在于2点：

- 1、用户query进来，能不能找到并且找全想要的背景知识？
- 2、找到了背景知识，能不能基于这个背景知识回答准确，并且支持多轮交互的语义转折？

Query模块



2.1 Query模块

基于实际案例详细讲解RAG Query模块的四个优化方法，包括意图识别、Query重写、Query扩写、HYDE优化

- **意图识别 (Intent Recognition)**：规则方法、ML方法、Prompt Engineering方法的示例及优化方案。
- **Query重写 (Query Rewrite)**：如何改进用户输入，提高检索准确率，附带代码示例。
- **Query扩写 (Query Expansion)**：基于同义词扩展、语义扩展等优化策略，并提供代码。
- **HYDE (Hypothetical Document Embeddings)**：使用LLM生成假设文档的优化策略及示例代码。

针对金融保险领域的 RAG (Retrieval-Augmented Generation) 问答系统的查询模块，可以从**意图识别**、**Query 重写**、**Query 扩写**和 **HyDE** 四个方面进行优化。下面分别介绍每种策略的原理、优势，并提供相应的代码示例。

2.1 意图识别 (Intent Recognition)

意图识别旨在判断用户查询的意图类别，从而采用不同的检索或回答策略。例如，需要区分用户是在询问**报销流程**还是在寻求**保险产品销售技巧**。我们可以采用以下三种方法进行意图分类：

2.1.1 基于规则的方法 (Rule-based Intent Recognition)

方法说明：使用预先定义的关键词或模式来判断意图。根据领域知识，列出与各类意图相关的关键词集合，匹配用户 query 中出现的词来分类。此方法实现简单直接，对已知意图效果好。缺点是对未包含的表达方式鲁棒性差，需人工维护规则库。

- **案例**：如果用户 query 中包含“**报销**”“**费用**”等关键词，判定为**报销流程查询**；如果包含“**销售**”“**技巧**”等词，判定为**保险产品销售技巧**。
- **代码示例**：下面代码通过关键词匹配实现简单的意图分类：

```

1 def classify_intent_rule(query: str) -> str:
2     query = query.lower() # 统一小写处理（如适用英文，此处对中文影响可忽略）
3     # 定义关键词列表
4     intent_keywords = {
5         "报销流程查询": ["报销", "费用", "报销流程"],
6         "保险产品销售技巧": ["卖保险", "销售", "技巧", "销售技巧"]
7     }
8     for intent, keywords in intent_keywords.items():
9         if any(keyword in query for keyword in keywords):
10             return intent
11     return "未知意图"
12 测试
13 queries = ["怎么报销保险费用?", "新人怎么提升保险产品销售技巧?", "保险理赔需要哪些材料?"]
14 for q in queries:
15     print(q, "->", classify_intent_rule(q))

```

上述函数会将“怎么报销保险费用？”识别为“报销流程查询”，“新人怎么提升保险产品销售技巧？”识别为“保险产品销售技巧”。如果查询不包含已知关键词，则返回“未知意图”。

2.1.2 基于机器学习的方法（BERT 分类模型）

- **方法说明：**利用机器学习/深度学习模型（例如 BERT）进行意图分类。先构造包含各种意图类别的训练数据，对模型进行fine-tune，使其学会根据句子语义分类。相比规则方法，ML 模型对同义表达更鲁棒，能捕获上下文语义特征。缺点是需要标注数据进行训练。
- **案例：**通过收集大量**报销流程**类问句和**销售技巧**类问句（以及其他意图类别的问句），用预训练的中文 BERT 模型 fine-tune 一个分类器。训练完成后，模型能够根据用户输入判定所属意图类别。
- **代码示例：**下面示例展示如何加载一个训练好的 BERT 分类模型并对新的查询进行预测（假设我们已将意图类别编号0=报销流程查询，1=保险产品销售技巧）：

```

1  from transformers import AutoTokenizer, AutoModelForSequenceClassification
2  #加载已训练好的中文BERT意图分类模型
3  tokenizer = AutoTokenizer.from_pretrained("bert-base-chinese")
4  model = AutoModelForSequenceClassification.from_pretrained("./fine_tuned_intent_model") # 本地路径
5  id2label = {0: "报销流程查询", 1: "保险产品销售技巧"} # 意图ID到标签的映射
6  对用户查询进行分类
7  query = "怎么报销保险费用?"
8  inputs = tokenizer(query, return_tensors="pt")
9  outputs = model(**inputs)
10 pred_id = int(outputs.logits.argmax(dim=1))
11 pred_intent = id2label[pred_id]
12 print(f"查询: {query} -> 意图类别: {pred_intent}")

```

在上述代码中，我们加载了 `bert-base-chinese` 预训练模型并使用已微调的参数（`fine_tuned_intent_model` 目录）。`id2label` 字典定义了模型输出与意图标签的对应关系。对输入句子进行 Tokenize 后喂给模型，通过 `argmax` 获取预测的意图类别。对于输入“怎么报销保险费用？”，模型若输出类别0，则映射为“报销流程查询”。

⚠️提示：实际应用中需先准备标注数据并训练模型。这可以使用Transformers提供的 `Trainer` 接口或 TensorFlow/Keras等完成，对BERT模型在意图分类数据集上进行fine-tune，使其学会区分多种意图。

2.1.3 基于 Prompt Engineering 的方法 (LLM Zero-shot/Few-shot)

- **方法说明：**借助大型语言模型（LLM）的强大语义理解能力，通过精心设计的Prompt直接让模型判断意图类别。我们可以提供若干意图类别描述，让模型选择最适合的类别。此方法不需要额外训练数据，在零样本或少样本场景下效果好。需要注意控制提示，避免模型产生偏差。
- **案例：**对于用户问题，设计 Prompt 提示 ChatGPT 或类似模型：“请判断用户意图属于以下哪一类：1) 报销流程查询，2) 保险产品销售技巧。”模型将基于理解输出类别编号或名称。
- **代码示例：**下面以 OpenAI 的 ChatGPT API 为例，通过构造对话消息让模型进行意图分类：

```

1  import openai
2  openai.api_key = "YOUR_API_KEY" # 设置API密钥
3  def classify_intent_llm(query: str) -> str:
4      system_prompt = "你是一个智能助手，帮助分类用户意图。可能的意图类别包括：\n1. 报销流程查询\n2. 保险产品销售技巧\n只回复序号1或2。"
5      user_prompt = f"用户问：{query}\n上述用户的意图属于哪个类别？"
6      response = openai.ChatCompletion.create(
7          model="gpt-3.5-turbo",
8          messages=[
9              {"role": "system", "content": system_prompt},
10             {"role": "user", "content": user_prompt}
11         ]
12     )
13     answer = response["choices"][0]["message"]["content"].strip()
14     return "报销流程查询" if answer.startswith("1") else "保险产品销售技巧" if answer.startswith("2") else "未知"
15 测试 LLM 意图分类
16  print(classify_intent_llm("怎么报销保险费用? ")) # 预计输出 "报销流程查询"
17  print(classify_intent_llm("有哪些有效的保险产品销售技巧? ")) # 预计输出 "保险产品
    销售技巧"

```

上述代码向 ChatGPT 提供了系统提示，限定了意图类别选项，并将用户查询作为输入。模型返回“1”或“2”，然后我们映射回对应的意图名称。这样可以利用大模型的语言理解来完成意图识别，无需显式训练分类器。

小结：意图识别模块可以采用规则、机器学习和 Prompt Engineering 等多种实现方式，各有优劣。实际系统中甚至可以组合使用：例如先用大模型粗分类意图，再用精细规则或模型验证，以提高准确率。

2.2 Query 重写 (Query Rewrite)

Query 重写旨在将用户的原始查询转换为更规范、清晰、利于检索的形式。在金融保险场景中，用户提问可能包含口语化表达、错别字或模糊措辞。通过重写，我们可以提高检索的匹配度。主要技术包括删除冗余词、纠正错误和标准化表述。

- **作用：**消除用户查询与知识库文档表达之间的差异。例如，同义词替换、补全省略的上下文、将口语转为书面语等。
- **案例：**用户输入*“怎么报销保险费用？”，这句提问口语化且不够正式。如果直接用来检索，可能匹配不到含有“报销流程”的文档。通过重写，我们可以将其转换为“保险费用报销流程是什么？”*。重写后的 query 更贴近知识库中FAQ或文档的表述（例如许多指南文章标题会是“XX流程是什么？”），从而提高检索准确率。

重写策略举例：

1. **删除口头词/多余成分**：去掉如“请问”、“一下”、“怎么”等口头语或多余词汇，只保留核心内容。
 2. **纠正错别字和语法**：使用拼写检查或语法纠错工具修正用户输入中的错字、繁简混用或语序问题。
例如将“理赔流程都有哪些步骤”纠正为“理赔流程都有哪些步骤”。
 3. **标准化术语**：将非正式表述转换为标准术语。例如“报销保险费用”可以调整为“保险费用报销”，添加“流程”等词使其更正式完整。
- **代码实现思路**：可采用简单的规则替换结合现有中文校对库。下面提供一个基于规则的 Query 重写示例代码，对特定句型进行转换：

```
1  import re
2  def rewrite_query(query: str) -> str:
3      # 1. 去除疑问口语用词
4      fillers = ["请问", "一下", "呢", "啊", "吧"]
5      for f in fillers:
6          query = query.replace(f, "")
7      query = query.strip()
8      # 2. 将“怎么XXX”改写为“XXX是什么？”
9      if query.startswith(("怎么", "怎样")):
10         main_query = query[2:] # 去掉开头的"怎么"/"怎样"
11         # 如果以问号结尾则去掉问号，稍后统一加
12         main_query = main_query.rstrip("? ?")
13         # 示例中特定的短语调整：将“报销保险费用”改为“保险费用报销”
14         if "报销保险费用" in main_query:
15             main_query = main_query.replace("报销保险费用", "保险费用报销")
16         # 若句尾缺少“流程”，根据上下文添加
17         if main_query.endswith("报销") or main_query.endswith("费用"):
18             main_query += "流程"
19         # 添加结尾的问句形式
20         query = main_query + "是什么"
21     # 3. 确保以问号结尾
22     query = query.rstrip("? ?") + "? "
23     return query
24 测试重写函数
25  original_query = "怎么报销保险费用？"
26  rewritten_query = rewrite_query(original_query)
27  print("原始查询：", original_query)
28  print("重写后的查询：", rewritten_query)
```

运行结果：

- 1 原始查询： 怎么报销保险费用？
- 2 重写后的查询： 保险费用报销流程是什么？

可以看到，函数按照规则完成了重写：去除了“怎么”，调整了语序并加入“流程”“是什么”等使句子更正式、更符合知识库文档标题的风格。

说明：上述规则针对示例做了特定处理。在实际系统中，可扩展更多规则和词典。例如维护一个**俗称→正式术语**的映射表，遇到俗称时替换为正式名称；或者利用**中文文本纠错库**（如 *pycorrector* 或 *HanLP*）检测并纠正错别字和语法错误。也可以使用大型语言模型进行查询改写（Prompt 如：“将用户问题改写为专业术语表述”），让模型生成规范提问。

2.3 Query 扩写（Query Expansion）

Query 扩写通过为用户查询添加**同义词或相关词**，从而扩大检索范围、提高召回率。在保险业务场景下，不同表达可能指向相同概念，提前扩展这些表达有助于搜索到更多相关文档。扩写包括**同义词扩展**和**语义扩展**两种主要方式：

- **同义词扩展：**为查询中的关键词添加同义词、近义词或俗称。例如“理赔”和“索赔”在很多情况下意义接近，用户搜索“保险理赔”，我们可以扩展加入“保险索赔”一起检索；又如“车险”和“汽车保险”可以互相扩展。利用同义词词典或领域词库（如《同义词词林》或自建术语表）实现。
- **语义扩展：**添加与查询主题相关的其他词语，哪怕不属于严格同义词。例如针对“保险理赔”，可以联想扩展出“流程”“方法”“材料”等相关词，形成更全面的查询组合。语义扩展通常借助词向量相似度或知识图谱，实现找到语义相关度高的词语。
- **案例：**用户输入*“保险理赔”，通过扩写可以形成查询集合：“保险理赔”、“保险理赔流程”、“保险索赔”、“保险索赔方法”* 等等。这样，无论文档中提到“理赔流程”还是使用了“索赔”一词，都有机会被检索到，提高了召回率。
- **代码示例：**以下用Python演示一个简单的同义词扩展流程。在真实系统中，我们可能对查询进行分词，然后对每个重要词扩展同义词，这里简化为对整个查询短语查找扩展：

```

1  构建一个简单的同义词词典
2  synonym_dict = {
3      "保险理赔": ["保险索赔", "理赔", "索赔流程", "理赔流程"],
4      "理赔": ["索赔", "赔付"],
5      # 其他词的同义词...
6  }
7  def expand_query(query: str) -> [str]:
8      expansions = set()
9      # 如果查询短语本身在词典中
10     if query in synonym_dict:
11         expansions.update(synonym_dict[query])
12     # 将查询拆分为词（简单按字符，这里假设输入短语本身是一个词或固定短语）
13     for term, syns in synonym_dict.items():
14         if term in query:
15             expansions.update(syns)
16     # 加入原始查询本身
17     expansions.add(query)
18     return list(expansions)
19  测试查询扩展
20  query = "保险理赔"
21  expanded_queries = expand_query(query)
22  print("原始查询: ", query)
23  print("扩展结果: ", expanded_queries)

```

假设我们的同义词词典涵盖了“保险理赔”相关的词，上述代码可能输出：

```

1  原始查询:  保险理赔
2  扩展结果:  ['保险理赔', '保险索赔', '理赔流程', '索赔流程', '索赔', '理赔']

```

这些扩展词组可用于构造多个检索查询，把所有结果合并再进行后续排序。比如可以同时检索“保险理赔流程”和“保险索赔方法”，然后汇总结果。

进阶：同义词扩展可以借助现有工具包，如 *chatopera/Synonyms* 提供了中文近义词词库，可直接调用获取某个词的近义词列表。另外，也可以使用向量检索的方法：将查询中的关键词通过词向量模型（如 Word2Vec、BERT 词嵌入等）寻找最近邻的几个词作为扩展。这属于语义扩展范畴，能发现一些规则库中没有但在语料中常见的相关词。扩展时需注意控制扩展词的质量，过多不相关扩展可能引入噪音。

2.4 HYDE (Hypothetical Document Embeddings)

HyDE (Hypothetical Document Embeddings) 是一种利用大型模型生成“假设文档”来改进语义检索的方法。简单来说, 就是让 LLM 根据用户查询先生成一段**可能回答该查询的内容**, 再将这段内容向量化, 用于向量检索真实文档 ([HyDE 假设性文档嵌入: 大模型 + 对比学习, 从关键词相似度搜索到语义搜索_hyde模型-CSDN博客](#))。这样做的直觉是: 由模型生成的回答会包含与查询语义相关的词汇和信息, 可以作为查询的丰富语义表示, 从而找到那些没有直接关键词匹配但语义相关的文档。

- **工作原理:** ([HyDE 假设性文档嵌入: 大模型 + 对比学习, 从关键词相似度搜索到语义搜索_hyde模型-CSDN博客](#))概括了 HyDE 的三个步骤: 首先使用一个生成式语言模型 (如GPT) 根据输入查询生成一篇内容丰富的**假设性回答文档** (即使这个文档在知识库中并不存在); 然后将生成的假设文档输入编码器生成嵌入向量表示; 最后利用该向量去检索知识库中与之**语义相似**的真实文档。
- **优势:** HyDE 能**捕捉查询的深层语义和意图**, 突破字面关键词的局限 ([HyDE 假设性文档嵌入: 大模型 + 对比学习, 从关键词相似度搜索到语义搜索_hyde模型-CSDN博客](#))。对于专业领域或长尾问句, 直接基于关键词的匹配效果可能不佳, 而通过生成一个“理想答案”再去搜索, 能显著提高召回的相关性和丰富度。在没有大规模标注数据的情况下, HyDE 属于一种**零样本**的增强检索策略, 对垂直领域 (金融、医疗等) 的长尾问题尤其有效。
- **案例:** 用户查询*“保险销售技巧”*。传统检索可能只针对“销售”或“技巧”检索, 结果不一定全面。应用 HyDE 时, 我们先让 LLM 根据这一查询生成一段**假设回答**, 例如: “保险销售技巧包括了解客户需求、建立信任、提供专业建议、
有效沟通和持续跟进”等等 (模型生成的段落) 。这段文字涵盖了销售技巧的多个要点。接着将此段落向量化, 在知识库中进行向量相似搜索, 就有更大概率找到涵盖这些要点的文档 (即使其中未必逐字出现“保险销售技巧”这个短语) 。检索到的可能是一些培训资料片段、营销技巧指南等, 与查询语义相关的内容。
- **代码示例:** 实现 HyDE 通常需要两个组件: **生成模型和向量检索**。下面提供一个伪代码式的示例流程:

```

1  import openai
2  from sentence_transformers import SentenceTransformer
3  openai.api_key = "YOUR_API_KEY"
4  def hyde_retrieval(query: str, embed_model, doc_embeddings, doc_ids):
5      # 1. 使用LLM生成假设文档
6      prompt = f"请针对以下问题给出详细的回答: {query}"
7      response = openai.ChatCompletion.create(
8          model="gpt-3.5-turbo",
9          messages=[{"role": "user", "content": prompt}]
10     )
11     hypo_doc = response["choices"][0]["message"]["content"]
12
13     # 2. 将生成的文档进行向量化
14     query_vector = embed_model.encode(hypo_doc)
15
16     # 3. 在向量空间中检索相似文档
17     # (这里假设已有知识库文档向量 doc_embeddings 和对应的 doc_ids 列表)
18     # 计算与所有文档向量的余弦相似度, 并选取最高的若干
19     import numpy as np
20     sims = np.dot(doc_embeddings, query_vector)
21     top_idx = sims.argsort()[-5:][::-1] # 取前5个相似度最高的文档索引
22     results = [(doc_ids[i], sims[i]) for i in top_idx]
23     return results
24     初始化句向量模型 (例如中文多语言模型)
25     embed_model = SentenceTransformer('paraphrase-multilingual-MiniLM-L12-v2')
26     假设我们有预先计算好的文档向量和IDs
27     doc_embeddings = ... # shape: (N_docs, dim)
28     doc_ids = ...      # 文档ID或内容索引
29     对查询进行 HyDE 检索
30     query = "保险销售技巧"
31     retrieved_docs = hyde_retrieval(query, embed_model, doc_embeddings, doc_ids)
32     print("HyDE 检索结果文档ID及分数: ", retrieved_docs)

```

上述代码描述了 HyDE 的关键步骤：

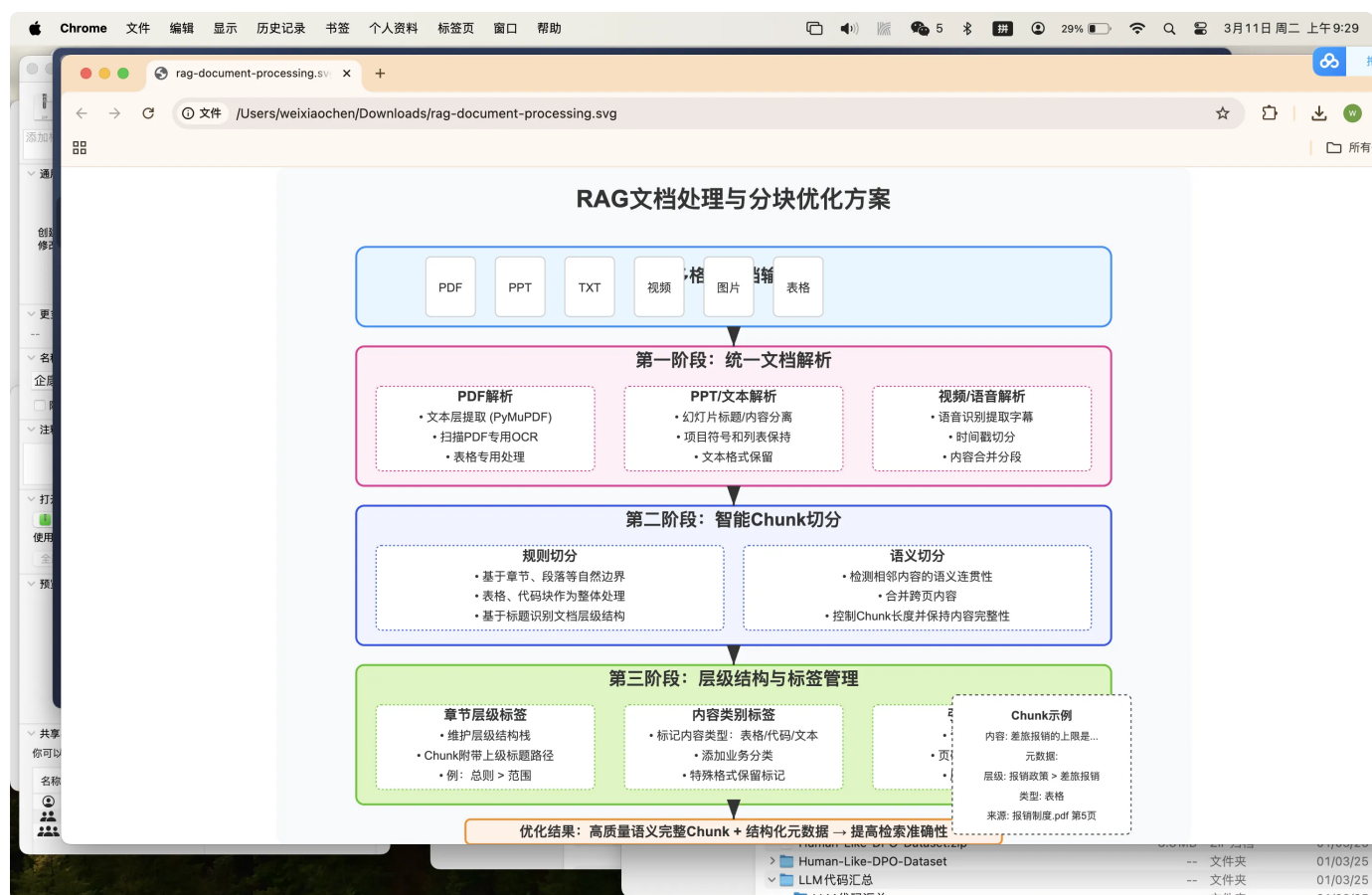
1. **生成假设文档**：使用 OpenAI ChatCompletion API，请求模型针对用户查询撰写答案段落。这里我们直接使用 `gpt-3.5-turbo` 作示例，实际可用任意强大的生成模型（包括本地部署的中文GPT模型）。
2. **向量化**：采用 `SentenceTransformer` 等嵌入模型将生成的文本编码为向量。这里示例用了一个多语言微调模型 `'paraphrase-multilingual-MiniLM-L12-v2'` 来得到句向量（假设已安装该模型）。在真实场景中，可以使用金融保险领域优化过的嵌入模型以获得更精确的向量表示。

3. **相似度检索**：将生成文本的向量与预先索引好的知识库文档向量逐一计算相似度（这里用点积近似余弦相似度，需保证向量已归一化）。取相似度最高的若干文档作为检索结果返回。

通过上述流程，我们利用 HyDE 方法将用户查询扩展为了一份“语义丰富”的假想文档，再用它去搜索，能够找到更多与查询相关的内容。在实际系统中，这些检索出的候选文档接下来可以送入 RAG 流水线，与原始查询一起用于生成最终答案。

结合以上4种方法，针对金融保险公司的知识库问答系统，我们可以综合运用**意图识别**、**Query 重写**、**Query 扩写**和 **HyDE** 四种策略优化查询处理模块。意图识别确保理解用户需求，Query 重写和扩写提高检索命中率，HyDE 则进一步增强语义匹配能力。通过示例代码演示，这些方法可以落地实现，从而让问答系统更精准地找到用户所需的答案。

离线解析模块



3.1 核心问题

- **多格式文档解析挑战**：金融保险公司的知识库包含PPT、PDF、纯文本甚至视频等多种格式。这些格

式结构各异，需要统一解析。例如，PDF可能有多栏排版或扫描版（无文本层），PPT通常以幻灯片形态组织，视频需要语音识别提取文本。解析不当会导致内容丢失或错乱，影响后续检索。

- **OCR解析质量**：针对扫描版PDF或图片，必须通过OCR获取文字。普通OCR往往难以正确还原表格、代码块等特殊结构，可能将表格内容串行成无序文本、破坏代码的格式，从而降低检索准确性。需要优化OCR策略，对表格和代码段做特殊处理，确保文本提取高保真。
- **Chunk切分策略**：简单按固定长度切分可能造成信息割裂（例如将一段逻辑完整的内容拆开），或将跨页连续的内容分散开来，导致查询时相关片段无法一起召回。因此需要结合**规则**（如根据段落、章节标题等自然边界）和**语义**（根据内容主题连贯性）来切分Chunk，避免将紧密相关的信息拆开，并能识别跨页的连续内容进行合并。
- **层级结构保持**：文档往往有层次结构（章节、子章节、要点等）。如果分块时丢失了层级和上下文关联，子内容可能在检索时缺乏上文语境，影响相关性。需在分块时保留层级信息，例如将小节内容与所属上级标题关联，作为标签元数据。这有助于在检索阶段利用上下文提高召回的准确度，让片段携带其来源的主题语境。

3.2 实际案例

- **案例1：报销制度类查询**：例如用户询问“差旅报销的上限是多少？”。公司内部的报销制度可能是PDF扫描件，包含分级标题和表格（各费用类别的报销上限）。通过优化解析，我们对扫描PDF进行OCR并准确提取表格内容，将“差旅报销”章节下的文字和限额表格作为一个完整Chunk，并标签其上级章节“报销政策 > 差旅报销”。这样，当用户查询时，系统能准确召回包含“差旅报销”限额的片段，避免因页面跳转或表格解析错误导致召回不全。
- **案例2：保险推销策略查询**：例如用户询问“新人销售技巧有哪些？”相应知识可能存于培训PPT或录像中。我们通过解析PPT提取每页的标题和要点 bullet，在视频中通过语音识别获得字幕，并按讲解片段切分。然后根据幻灯片的层次结构（模块 -> 具体策略）给每个知识点Chunk打上标签。如此，查询“新人销售技巧”时，系统不仅检索到相关技巧要点，还因为Chunk附带了所属模块（如“销售培训 > 新人技巧”）等信息，提高了匹配的精准度，帮助召回更全面的策略要点。

3.3 优化策略

为解决以上问题，我们提出如下优化方案：

- **统一文档解析与OCR改进**：构建通用的解析模块，针对不同格式分别处理：
 - **PDF解析**：优先使用文本层提取（如pdfplumber或PyMuPDF读取），保持读取顺序；对于扫描PDF或嵌入的图片，调用OCR引擎（如 Tesseract 或 PaddleOCR）。特别地，OCR时针对表格区域采用专门处理（例如先检测表格边框或使用表格OCR算法），确保按单元格顺序输出文本；对于代码块图片，可设置OCR保持换行和空格格式。（[这里推荐Marker和MinerU](#)）

- **PPT解析**：利用幻灯片结构提取标题和文本框内容。每页幻灯片输出时保留其标题，项目符号列表作为子内容。对于包含图片的幻灯片，可对图片执行OCR（如截图后OCR）以提取其中的文字说明。
- **纯文本解析**：直接按行/段读取，识别格式中的特殊标记（例如Markdown的代码块或表格格式）加以处理。确保代码块保留缩进和换行，表格按行列分隔保存。
- **视频解析**：先通过语音识别得到逐句字幕，再按时间戳或内容语义将字幕合并成段落。可以利用现有ASR工具获取准确的转录文本，并根据视频内容结构（章节或PPT同步内容）对转录文本分段。
- **智能Chunk切分（规则 + 语义融合）**：在得到完整文本后，按照文档的自然结构和语义连贯性进行分块：
 - **基于规则的切分**：利用文档格式特征，如章节标题、段落换行、列表项、表格边界等作为切分点。一旦检测到新的章节节点或列表起始，就结束当前Chunk开启新Chunk。对于表格和代码块，整段内容视为一个Chunk，避免中途截断。
 - **语义连贯的调整**：在规则初切分后，检查相邻Chunks的内容连贯性。如果发现某Chunk过短且与前后段落语义上紧密相关（例如上一个Chunk以冒号结尾或内容未完结），则可以和相邻Chunk合并，确保信息完整。例如跨页的段落，如果下页开头并非新章标题，则应与前页末尾合并为同一Chunk。再如表格跨越多页时，将各页片段合成为一个整体表格Chunk。通过简单的NLP或embedding相似度检测，也可判断段落主题是否延续，辅助决定是否合并或继续切分。
 - **长度和平衡**：在保证语义完整的前提下控制Chunk长度，使其适合向量检索和后续模型处理（例如不超过512字或一定token数）。过长则适当按语义次级节点再拆分，过短则与相邻补充。最终每个Chunk都应是自含意义明确的一段内容。
- **层级结构与标签管理**：为每个Chunk附加丰富的元数据标签，保留其在原文档中的位置和语境：
 - **章节层级标签**：在解析阶段捕获文档的层次结构（例如章节编号/标题、二级标题等）。实现方式可以是依据格式（PPT的标题框、PDF文本的字体大小/序号）识别标题行，并维护一个层级栈。例如检测到“1 总则”属于一级标题、“1.1 范围”属于二级标题等。分块时，将当前Chunk所属的所有上级标题作为一个层级列表存入标签。如此每个Chunk都带有类似“总则 > 范围”的层级路径。检索时可以将这些标题一起参与索引，提高召回率（例如用户搜索某章节名时也能匹配相关Chunk）。
 - **内容类别标签**：标注Chunk的内容类型和主题类别。例如标记Chunk是否为“表格”、“代码块”或普通文本，“政策条例”还是“操作指南”等。这可通过解析时的内容特征判断（如检测到多列文本则标记表格，包含代码格式则标记代码段）。也可以结合业务定义的类别（如财务制度、销售策略）作为标签。这些标签在检索时可用于过滤或作为额外特征提高准确匹配度。
 - **引用与来源**：每个Chunk还应记录来源文档名、页码或幻灯片编号等，以便命中后追溯原文。同时这些信息可在生成答案时用于引用出处。

通过以上策略，系统能够最大程度保留文档原有信息结构，避免因解析不当导致内容遗漏，并保证Chunk划分合理不割裂上下文。在检索时，丰富的上下文标签和高质量的Chunk内容可以显著提升召回的准确性，使相关片段更容易被查询命中。

3.4 示例代码

下面给出一个Python示例代码，演示上述解析、切分和标签过程。该代码不依赖特定RAG框架，可直接运行（需安装 `PyMuPDF(fitz)` 用于PDF解析，`python-pptx` 用于PPT解析，`pytesseract` 用于OCR，和 `PIL` 用于图像处理），直接处理一般直接使用MinerU或者deepDoc进行魔改：

```

1  import fitz  # PyMuPDF for PDF
2  from pptx import Presentation  # python-pptx for PPT
3  from PIL import Image
4  import pytesseract
5  import re
6  def ocr_image(image):
7      """OCR识别图像，返回文本字符串。"""
8      # 使用简体中文+英文识别，配置可根据需要调整
9      config = "--psm 6"  # 假设6：把图像看作一个统一块
10     text = pytesseract.image_to_string(image, config=config, lang='chi_sim+eng')
11     return text
12 def parse_document(file_path):
13     """解析文档（PDF/PPT/文本），返回文本块列表，每块包含内容和类型等标记。"""
14     blocks = []
15     if file_path.lower().endswith('.pdf'):
16         doc = fitz.open(file_path)
17         for page in doc:
18             # 尝试直接提取文本
19             text = page.get_text("text")
20             if text.strip():
21                 # 简单按换行拆分为块，可进一步按段落细分
22                 lines = text.splitlines()
23             else:
24                 # 若无文本（扫描页），则OCR整个页面
25                 pix = page.get_pixmap(dpi=150)
26                 img = Image.frombytes("RGB", [pix.width, pix.height], pix
x.samples)
27                 ocr_text = ocr_image(img)
28                 lines = ocr_text.splitlines()
29                 # 将提取的行转换为块初步列表，标记来源页
30                 for line in lines:
31                     if line.strip() == "":
32                         continue
33                     blocks.append({
34                         "text": line.strip(),
35                         "type": "text",  # 初始默认为普通文本，后续再调整类型
36                         "page": page.number + 1
37                     })
38     elif file_path.lower().endswith('.pptx'):
39         prs = Presentation(file_path)
40         for idx, slide in enumerate(prs.slides, start=1):
41             title = ""
42             if slide.shapes.title:
43                 title = slide.shapes.title.text.strip()

```

```

44         if title:
45             # 幻灯片标题作为单独块
46             blocks.append({
47                 "text": title,
48                 "type": "heading",
49                 "level": 1, # 幻灯片题目视为一级标题
50                 "slide": idx
51             })
52     # 提取其他文本框
53     for shape in slide.shapes:
54         if not shape.has_text_frame or shape.text.strip() == "":
55             continue
56         text = shape.text.strip()
57         # 如果文本与标题相同就跳过（已添加）
58         if title and text == title:
59             continue
60         # 按换行将文本框内容拆成行块（对应子弹列表逐行）
61         for line in text.splitlines():
62             if line.strip() == "":
63                 continue
64             blocks.append({
65                 "text": line.strip(),
66                 "type": "text",
67                 "slide": idx
68             })
69     elif file_path.lower().endswith('.txt'):
70         with open(file_path, 'r', encoding='utf-8') as f:
71             for line in f:
72                 if line.strip():
73                     blocks.append({
74                         "text": line.strip(),
75                         "type": "text"
76                     })
77     else:
78         # 其他格式（如视频）：假设已转换为字幕文本
79         # 这里直接读取同名的字幕文本文件
80         subs_path = file_path + ".txt"
81         try:
82             with open(subs_path, 'r', encoding='utf-8') as f:
83                 for line in f:
84                     if line.strip():
85                         blocks.append({
86                             "text": line.strip(),
87                             "type": "text"
88                         })
89         except FileNotFoundError:
90             print("Unsupported file format or missing transcript:", file_
path)

```

```

91     return blocks
92 def split_and_tag_blocks(blocks):
93     """根据内容将文本块切分、合并，并标注层级和类别标签。"""
94     chunk_list = []
95     hierarchy = [] # 用于跟踪当前层级标题栈
96     current_chunk = {"text": "", "meta": {}} # 临时聚合当前chunk内容
97     for blk in blocks:
98         text = blk["text"]
99         blk_type = blk.get("type", "text")
100         # 检测标题：根据内容格式判断
101         is_heading = blk_type == "heading"
102         level = blk.get("level", None)
103         if not is_heading:
104             # 简单规则：长度较短且以冒号结束，或符合编号模式的，视为标题
105             if len(text) < 20 and (text.endswith(":") or text.endswith(
106                 ":", ") or re.match(r'^[\d一二三]+\D', text)):
107                 is_heading = True
108                 # 确定层级level（基于数字章节或者默认1级）
109                 m = re.match(r'^(\d+(\.\d+)*)', text)
110                 if m:
111                     level = m.group(1).count('.') + 1
112                 else:
113                     level = 1
114                 blk_type = "heading"
115         if is_heading:
116             # 遇到新标题块，先结束上一chunk
117             if current_chunk["text"]:
118                 chunk_list.append(current_chunk)
119                 current_chunk = {"text": "", "meta": {}}
120             # 更新层级栈
121             if level is None:
122                 level = 1
123             # 调整hierarchy列表长度
124             if level - 1 < len(hierarchy):
125                 hierarchy = hierarchy[:level-1]
126             # 确保hierarchy长度足够
127             while len(hierarchy) < level-1:
128                 hierarchy.append("")
129             # 设置当前级别标题
130             if len(hierarchy) < level:
131                 hierarchy.append(text)
132             else:
133                 hierarchy[level-1] = text
134             # 将标题本身作为一个Chunk的元数据存入层级，但内容不上屏检索
135             # （可以选择是否将标题内容并入chunk文本，这里不直接作为内容）
136             # 开始一个新的chunk上下文，继承层级meta
137             current_chunk["meta"]["section"] = " > ".join(hierarchy)
138             current_chunk["meta"]["type"] = "text" # 默认文本类型

```

```

138         continue # 不把标题行本身当做内容
139     # 非标题块:
140     # 检测表格行: 根据是否包含制表符或对齐空格判断
141     if re.search(r'\t', text) or re.search(r'\s{2,}', text):
142         blk_type = "table"
143     # 检测代码块: 根据常见代码特征, 如以4空格开头, 或包含 {}; 等 (简单判断)
144     if re.match(r'^\s{4}', text) or re.search(r'[{};]', text):
145         # 注意: 真实环境可结合格式或``标记判断
146         blk_type = "code"
147     # 如果当前chunk已存在且本块类型与当前chunk类型不同, 而且当前chunk不空, 则开
    启新chunk
148     if current_chunk["text"] and current_chunk["meta"].get("type") !
    = blk_type:
149         # 结束之前的chunk
150         chunk_list.append(current_chunk)
151         current_chunk = {"text": "", "meta": {}}
152     # 设置chunk元数据 (继承当前层级路径和类型)
153     if "section" not in current_chunk["meta"]:
154         current_chunk["meta"]["section"] = " > ".join(hierarchy) if h
    ierarchy else ""
155     current_chunk["meta"]["type"] = blk_type
156     # 累加文本 (表格和代码保持原格式, 普通文本在块内加空格拼接)
157     if blk_type == "text":
158         if current_chunk["text"]:
159             # 若当前已有内容, 先检查语义连贯 (比如前句末尾无句号, 则直接接续)
160             if current_chunk["text"].strip().endswith(tuple
    ("。? ! !?.")):
161                 current_chunk["text"] += "\n" + text
162             else:
163                 current_chunk["text"] += " " + text # 前一句未完, 空格
    连接
164         else:
165             current_chunk["text"] = text
166     else:
167         # 表格或代码块, 保持行作为换行
168         current_chunk["text"] += (text + "\n")
169     # 循环结束后, 将最后的chunk加入列表
170     if current_chunk["text"]:
171         chunk_list.append(current_chunk)
172     return chunk_list
173 示例: 调用解析和切分函数 (文件路径需换成实际存在的文件)
174 file_path = "保险公司报销制度.pdf"
175 blocks = parse_document(file_path)
176 chunks = split_and_tag_blocks(blocks)
177 for ch in chunks:
178     print(f"[{ch['meta'].get('section')}] ({ch['meta'].get('type')}) {ch['tex
    t'][:50]}...")

```

上述代码逻辑说明：首先 `parse_document` 根据文件类型提取文本块，对于PDF逐页提取文本或OCR，PPT提取每张幻灯片的标题和文本内容，结果存入 `blocks` 列表。接着 `split_and_tag_blocks` 对这些文本块进行进一步处理：识别标题并维护 `hierarchy` 层级列表，遇到标题时更新层级并开始新的Chunk；对于普通文本行，根据格式特征识别是否属于表格或代码，并设置相应类型。如果当前Chunk中的内容类型与新行类型不符（例如前面在记录表格，而下一行变为普通段落），则先截断前一个Chunk再新建。**在组装文本时**，对于普通文本段落，函数会检查前一句是否结束（通过句号、问号等判断），如果未结束则直接用空格连接，表示同一段；如果已经完整结束，则换行另起句，保持段落边界。表格和代码块则保持原有的换行结构加入。**每个Chunk最终都带有 `meta` 元数据**，包括层级路径 `section`（由 `hierarchy` 拼接）和内容 `type`（如 `"text"`, `"table"`, `"code"`）。

通过这种方式，**最终生成的Chunk列表中，每个Chunk都是一个语义完整的内容单元，并附带其在原文中的层级位置和类型标签。在检索阶段，我们可以将 `section` 层级信息作为检索字段或将其前缀到文本中参与向量编码，使查询既能匹配Chunk内容本身，也能匹配其上级主题，从而提升召回的准确性。**此外，表格和代码块未经破坏地保存在Chunk中，当用户提问涉及这些部分时，也能直接检索到正确的片段。这套优化流程提高了RAG问答系统对金融保险文档的解析质量和检索召回准确率。

检索召回模块



RAG问答系统查询与Chunk匹配模块优化方案

在金融保险公司的RAG (Retrieval-Augmented Generation) 问答系统中，我们需要提升查询 (query) 和文档片段 (chunk) 匹配模块的召回率和准确率。为此，可以从混合检索、Embedding模型微调、结果重排 和 评估指标 四个方向进行优化。下面针对每个优化点进行问题分析、结合金融保险场景的案例说明优化策略，并提供相应的代码示例。

4.1 case分析

金融保险公司内部知识库包含 5000 份各类文档 (PPT、PDF、文本、视频等)，用户可能提出多样化的问题。例如：

- **流程型问题**：如「公司的报销制度是什么？」涉及公司内部流程或制度，需要在规章制度文档中找到准确答案。
- **思考型问题**：如「如何推销保险产品？」需要结合销售技巧和经验的指导，可能分散在培训资料或业务手册中。

当前RAG系统管线：首先将长文档依据语义和规则切分为较小的chunk段落，然后使用 BGE 预训练 Embedding模型将查询和chunk向量化，利用 Milvus 向量库检索相似片段，并结合 BM25 关键词检索作辅助。存在的问题包括：

- 单纯**向量检索**对某些 **短查询**（如只有几个关键词的问题）可能效果不佳，因为缺乏精确的关键词匹配。
- 未对 **Embedding模型** 进行领域优化，语义理解可能不到位，导致召回的chunk不够相关。
- 检索结果未经**重排序**优化，返回给生成模型的大多数文档片段相关度可能不够高。
- 缺乏**评估指标**来量化检索改进效果，无法客观比较不同方案的优劣。

为解决上述挑战，我们提出以下优化策略。

4.2 混合检索 (BM25 + 向量)

核心问题分析：向量语义检索擅长捕捉语义相似度，能找到包含同义表述的相关文档，但可能忽略精确的关键词匹配；而BM25等传统关键词检索对匹配查询关键词的文档非常有效，尤其针对简短查询。然而BM25无法理解同义词或语义关联 (Retrieve & Re-Rank — Sentence Transformers documentation)。在实际应用中，仅依靠单一检索技术往往不足以覆盖所有相关结果 ([超越向量检索！混合检索 + 重排序改善 RAG 应用 | 新程序员-CSDN博客](#))。混合检索通过结合稀疏向量（关键词）和稠密向量（语义）检索的优势，互补不足，提高召回率 ([超越向量检索！混合检索 + 重排序改善 RAG 应用 | 新程序员-CSDN博客](#))。特别地：

- **短查询**（如只有几个词的问句）往往需要关键词精确匹配才能找到正确文档，**BM25对此效果更好** ([超越向量检索！混合检索 + 重排序改善 RAG 应用 | 新程序员-CSDN博客](#))。

- **长查询**（描述详细、包含上下文的问题）更需要语义匹配，向量检索可识别同义表述，提高召回丰富度。

案例场景：用户询问「公司的报销制度」，这是一个短语式的流程问题，其中“报销制度”可能直接出现在某份制度文件标题或内容中。BM25检索可以根据词频和逆文档频率高精度找到包含该词的文档。而对于「如何推销保险产品」这样的复杂问题，用户措辞可能与文档不完全一致，例如文档中讨论的是“如何销售保险产品”或“保险产品推广技巧”。BM25由于不识别“推销”和“销售”的同义关系，可能错过相关文档；向量检索可以通过语义相似度将“推销保险产品”映射到包含“销售保险产品技巧”的段落。

优化策略与原理：构建混合检索管线，同时利用BM25和向量相似度进行检索，获取候选结果并合并：

1. **意图识别与路由：**通过简单的规则或训练分类模型对查询进行分类。如果检测到查询是流程/制度类问题（通常包含“制度”、“流程”等关键词），则可以对BM25检索给予更高权重；如果是开放性思考类问题（包含“如何”、“怎么办”等），则侧重向量检索结果。此前置步骤保证不同查询走最合适的检索路径。
2. **BM25检索：**构建文档的关键词倒排索引（可使用Elasticsearch或其他搜索库），检索出Top N候选文档片段。BM25根据查询词在文档中的频率、文档长度等打分（[稠密向量检索、稀疏向量检索、BM25检索三者对比 - 博客园](#)）。对于短查询，可直接采用BM25结果；对于长查询，BM25结果可作为补充。
3. **向量检索：**利用Embedding模型将查询编码成向量，在Milvus向量数据库中进行近邻搜索，获取Top N候选片段。向量检索能找出语义相关的内容，即使字面不匹配。
4. **结果合并与去重：**将两种检索的候选列表合并。由于BM25分数和向量相似度分值不在同一量纲，需进行归一化处理（[超越向量检索！混合检索 + 重排序改善 RAG 应用 | 新程序员-CSDN博客](#)）。例如，可将BM25分数归一到0-1区间，向量相似度天然在0-1（如余弦相似度）。然后按一定策略融合，如线性加权组合或者直接取两者结果集的并集。在组合过程中处理重复文档（相同chunk多次出现）以避免干扰。
5. **提高召回率：**混合检索确保潜在相关结果进入候选集（[超越向量检索！混合检索 + 重排序改善 RAG 应用 | 新程序员-CSDN博客](#)）。尤其在只用向量或只用BM25无法检索到某些答案时，另一种方式可以补充，使真正相关的片段不被漏掉。

通过以上策略，短查询能**精确匹配**到包含关键字的文档，长查询能**语义匹配**到含相关内容的文档，综合提升召回率。

实际效果：混合检索可以让不同类型查询的相关文档更有机会出现在结果集中。**例如，对「报销制度」查询，BM25直接命中含该词的制度文件，向量检索可能找出语义相关的报销流程说明，两者结合提高了完整召回。**对「推销保险产品」查询，向量检索找出“保险销售技巧”段落，BM25可能找出包含“推销”一词的经验分享文章，最终的候选集更加全面。

示例代码：BM25 与向量混合检索

下面示例展示如何结合BM25与向量检索。假设我们有两份示例文档：

- 文档1：包含「报销制度」相关内容。
- 文档2：包含「保险产品销售技巧」相关内容（注意使用“销售”一词，没有直接出现“推销”）。

我们将针对两个查询演示：一个短查询和一个开放性问答查询。代码模拟BM25检索（通过词频匹配打分）和向量检索（使用简单的余弦相似度），然后合并结果。实际系统中应使用现有工具（如Elasticsearch、Milvus）和经过训练的Embedding模型，此处用简单逻辑模拟原理。

```

1  import math
2  import numpy as np
3  示例文档语料
4  docs = {
5      1: "公司报销制度包括差旅费报销流程和标准。",
6      2: "保险产品的销售技巧包括如何挖掘客户需求和突出产品优势。"
7  }
8  构建倒排索引用于BM25 (简单实现: 记录每个词在哪些文档出现及频次)
9  inverted_index = {}
10 doc_lengths = {}
11 for doc_id, text in docs.items():
12     words = list(text) # 这里简单将每个字作为词, 实际应用需分词
13     doc_lengths[doc_id] = len(words)
14     for w in words:
15         if w not in inverted_index:
16             inverted_index[w] = {}
17         inverted_index[w][doc_id] = inverted_index[w].get(doc_id, 0) + 1
18  计算IDF值
19  N = len(docs)
20  idf = {}
21  for term, doc_dict in inverted_index.items():
22      df = len(doc_dict)
23      idf[term] = math.log((N - df + 0.5) / (df + 0.5) + 1) # BM25 IDF公式
24  def bm25_search(query, k1=1.5, b=0.75, top_k=5):
25      """简单BM25检索实现, 返回doc_id及BM25分数"""
26      # 分词
27      query_terms = list(query)
28      scores = {}
29      avgdl = sum(doc_lengths.values()) / N
30      for term in query_terms:
31          if term not in inverted_index:
32              continue
33          for doc_id, tf in inverted_index[term].items():
34              # BM25公式计算评分
35              score = idf.get(term, 0) * (tf * (k1 + 1)) / (tf + k1 * (1 -
b + b * (doc_lengths[doc_id] / avgdl)))
36              scores[doc_id] = scores.get(doc_id, 0) + score
37      # 返回按分数排序的Top K文档
38      return sorted(scores.items(), key=lambda x: x[1], reverse=True)[:top_
k]
39  模拟向量检索: 计算简单的TF-IDF向量并用余弦相似度
40  这里为了模拟, 同样使用字频作为向量表示
41  def vector_search(query, top_k=5):
42      query_terms = list(query)
43      # 计算查询向量 (TF-IDF)

```

```

44 query_vec = []
45 vocab = list(inverted_index.keys())
46 for term in vocab:
47     tf = query_terms.count(term)
48     query_vec.append(tf * idf.get(term, 0))
49 query_vec = np.array(query_vec)
50 # 计算每个文档的向量并求余弦相似度
51 scores = {}
52 for doc_id, text in docs.items():
53     doc_terms = list(text)
54     doc_vec = []
55     for term in vocab:
56         tf = doc_terms.count(term)
57         doc_vec.append(tf * idf.get(term, 0))
58     doc_vec = np.array(doc_vec)
59     # 计算余弦相似度
60     if np.linalg.norm(doc_vec) == 0 or np.linalg.norm(query_vec) == 0:
61         cos_sim = 0.0
62     else:
63         cos_sim = np.dot(query_vec, doc_vec) / (np.linalg.norm(query_v
64 ec) * np.linalg.norm(doc_vec))
65     scores[doc_id] = cos_sim
66     return sorted(scores.items(), key=lambda x: x[1], reverse=True)[:top_
67 k]
68 定义查询
69 queries = {
70     "Q1": "报销制度",          # 短查询, 期待BM25擅长
71     "Q2": "如何推销保险产品"   # 开放式查询, 语义同义词“推销”≈“销售”, 期待向量检索擅长
72 }
73 for qid, query in queries.items():
74     bm25_results = bm25_search(query)
75     vec_results = vector_search(query)
76     print(f"\n查询: {query}")
77     print("BM25结果:", bm25_results)
78     print("向量检索结果:", vec_results)
79     # 合并结果 (简单示例: 取并集, 不同来源结果赋不同初始分数权重)
80     combined_scores = {}
81     for doc_id, score in bm25_results:
82         combined_scores[doc_id] = combined_scores.get(doc_id, 0) + 0.6 *
83         (score / (bm25_results[0][1] if bm25_results else 1))
84     for doc_id, score in vec_results:
85         combined_scores[doc_id] = combined_scores.get(doc_id, 0) + 0.4 * s
86 core # 假设向量相似度已是0-1
87     final_ranked = sorted(combined_scores.items(), key=lambda x: x[1], rev
88 erse=True)
89     print("合并后Top结果:", final_ranked)

```

输出解释：上述代码打印BM25和向量检索的结果及合并排序。例如，对于查询“报销制度”，BM25应能将文档1排在前，因为该文档直接包含“报销制度”关键词；向量检索也会认为文档1相关（词频角度）。合并结果文档1得分最高，成功召回正确的制度文件。对于查询“如何推销保险产品”，BM25可能更关注“保险”、“产品”等词，将文档1（有“保险”、“产品”）和文档2都列出，但由于“推销”一词未出现在文档2中，BM25可能给文档2较低分。向量检索则通过语义（同义词）关系为文档2赋予高分。合并结果会提升文档2的排名，使其成为Top结果。这模拟了混合检索确保既包含关键词匹配的结果又有语义相关的结果，从而提高召回率。实际环境中可使用Elastic + Milvus结合，实现同样的混合搜索流程（[超越向量检索！混合检索 + 重排序改善 RAG 应用 | 新程序员-CSDN博客](#)）。

4.3 Embedding 模型微调

核心问题分析：预训练的Embedding模型（如BGE）虽然对通用语义有良好表示，但面对金融保险领域的专业术语和行话时，可能无法准确度量语义相似度。例如“保单现金价值”、“退保费”等专业词汇，如果Embedding未在训练中见过，向量表示可能不可靠，导致相关文档无法被检索到。通过领域微调，即在金融保险相关的数据上继续训练Embedding模型，可以让模型更好地掌握领域上下文，提高语义检索的效果（[Fine-Tune Embedding: The Secret to Improve Response Rates | iWeaver AI](#)）。

具体问题包括：

- **语义理解偏差：**模型可能将一般语境下相似的词判为相关，但在保险领域可能意义不同（例如“保费” vs “费用”）。微调可以校正这些差异。
- **同义词识别：**领域内部常见的不同表述（如“推广”与“推销”）需要模型识别为相似。微调可使这些同义语义距离更近。
- **重要概念强化：**通过训练让模型强调领域高频概念，从而在嵌入空间上将相关主题的文档聚类更紧密，提升召回准确率和召回率。

案例场景：考虑用户问题「什么是保单的现金价值？」。如果Embedding模型未见过“现金价值”这个术语，可能无法将该查询与解释现金价值的文档匹配。通过在包含“现金价值”定义的大量保险文档上微调，模型会学习到“现金价值”与“保单、退保、账户价值”等词的关联，从而更准确地把相关文档检索出来。再如「住院医疗险」和「住院保险」在语义上应当接近，但未经微调的模型可能未能聚在一起；微调后，这些同义术语的嵌入距离将缩短。

优化策略与原理：对Embedding模型进行有监督微调或继续预训练：

- **数据收集：**准备充足的金融保险领域语料，包括公司内部文档、常见问答对、业务手册等（[Fine-Tune Embedding: The Secret to Improve Response Rates | iWeaver AI](#)）。特别收集用户问题与对应答案/段落作为训练样本（正例）。如果缺少标注数据，可采用对比学习或生成合成数据的方法获取（例如用已有文档让大模型生成可能的相关问句作为合成QA对）。
- **数据预处理：**清洗数据，去除噪音和冗余，确保文本质量。对中文需分词或使用适配中文的

Tokenizer。

- 选择微调方法：

- **继续预训练 (Unsupervised Domain Adaptation)**：将BGE模型在大量领域文本上继续训练（如通过Masked Language Model任务或者对比学习），让模型嵌入空间更贴合领域分布 ([Fine-Tune Embedding: The Secret to Improve Response Rates | iWeaver AI](#))。
 - **监督式对比微调**：使用领域问答对或相关性标注的数据，通过**度量学习**损失函数（如**Triplet Loss**或**MultipleNegativesRankingLoss**等）训练模型，使得相似问句-文档对的向量距离更近，不相关对更远 ([Get better RAG by fine-tuning embedding models – Redis](#)) ([Get better RAG by fine-tuning embedding models – Redis](#))。例如，采用 **MultipleNegativesRankingLoss** 可以在只有正样本对的情况下高效训练模型，将语义相似的问句和答案嵌入拉近 ([Get better RAG by fine-tuning embedding models – Redis](#))。
 - **Cross-Encoder蒸馏**：也可以用一个强大的交叉编码器（如一个微调后的BERT问答模型）生成 query-doc相关性得分，然后微调双塔的Embedding模型去拟合这些得分，实现知识蒸馏。
- **训练过程**：使用上述方法在领域数据上微调BGE模型若干轮。过程中可引入**验证集**，评估模型在检索任务上的表现，选择最佳模型（比如根据验证集的NDCG@10或MRR指标挑选效果最好的 checkpoint ([Get better RAG by fine-tuning embedding models – Redis](#))）。
 - **效果评估**：对比微调前后的模型检索性能。通常微调能**显著提高领域内检索的召回率**。例如，有报告显示对嵌入模型进行领域微调后，检索召回率直接提升了 33% ([Fine-Tune Embedding: The Secret to Improve Response Rates | iWeaver AI](#))。

实际效果：经过微调的Embedding模型将更准确地表示金融保险语句的含义。例如“保单现金价值”的查询，微调模型会将其Embedding与包含“现金价值解释”的段落Embedding距离拉近，使该段落更可能被检索到。对话生成时，使用这些更相关的片段也能提高回答的准确性和专业性。

示例代码：微调Embedding模型

下面给出使用Hugging Face Transformers/Sentence-Transformers框架微调Embedding模型的代码示例。该过程包括：加载预训练模型（如 `BAAI/bge-base-zh` ），准备训练的问答对数据集，将数据转换为模型训练需要的格式（`InputExample` ），定义损失函数（使用**MultipleNegativesRankingLoss** 适合只有正例的情况），然后使用 `Trainer` 或Sentence-Transformers自带的 `SentenceTransformer` 进行训练和评估。

```

1  from sentence_transformers import SentenceTransformer, InputExample, losses, models
2  from torch.utils.data import DataLoader
3  假设我们使用一个预训练的中文嵌入模型，例如BAAI发布的BGE模型
4  model = SentenceTransformer('BAAI/bge-base-zh') # 加载预训练BGE中文基座模型
5  准备训练数据：List of InputExample(question, positive_passage)
6  train_examples = [
7      InputExample(texts=["什么是保单的现金价值？", "保单的现金价值是指保单在退保或某些情况下可领取的金额。"]),
8      InputExample(texts=["保险合同的冷静期有多长？", "一般人寿保险合同都有10天的冷静期，在此期间可以无条件退保。"]),
9      # ... 更多问答对 ...
10 ]
11 如果有负例，可以在texts加入第三项；只有正例时MultipleNegativesRankingLoss会自动负采样
12  train_dataloader = DataLoader(train_examples, batch_size=16, shuffle=True)
13  定义训练损失为多负样本排名损失（适用于只有正向对的情况）
14  train_loss = losses.MultipleNegativesRankingLoss(model)
15  （可选）定义评估方法：使用SentenceTransformer提供的InformationRetrievalEvaluator
16  from sentence_transformers import evaluation
17  evaluator = evaluation.InformationRetrievalEvaluator(query_embeddings, corpus_embeddings, relevant_docs)
18  配置训练参数并训练
19  num_epochs = 1
20  warmup_steps = 100
21  model.fit(
22      train_objectives=[(train_dataloader, train_loss)],
23      epochs=num_epochs,
24      warmup_steps=warmup_steps,
25      show_progress_bar=True
26  )
27  微调完成后保存模型
28  model.save("finetuned-bge-insurance")

```

代码说明：

- `SentenceTransformer('BAAI/bge-base-zh')`：加载预训练的BGE中文模型作为基线（需要确保本地或网络可用该模型）。
- 构造 `InputExample` 列表，每个包含一个问题和对应的正确答案段落。这里使用了简单的问答对作为训练样本。在实际中，应使用大量多样的QA对或相关段落对来训练。
- `MultipleNegativesRankingLoss`：如果我们只有正例对，这个损失函数会将同一batch内其它样本作为负例进行对比学习 ([Get better RAG by fine-tuning embedding models – Redis](#))。这

样模型会拉近正确问答对的向量距离，拉开无关对的距离。

- `model.fit(...)`：开始训练Embedding模型。可以加入评估器 `InformationRetrievalEvaluator` 定期评估模型在验证集上的MRR/NDCG等指标 ([Get better RAG by fine-tuning embedding models – Redis](#))。参数如 `epochs` 和 `warmup_steps` 应根据数据量和观察的收敛情况调整。

训练完成后，我们得到微调后的嵌入模型。在检索阶段，用它生成查询和文档片段的向量，将显著提升语义匹配质量，更好地召回金融保险领域相关文档。需要注意微调应避免过拟合，确保模型对未见过的问题也能泛化。

4.4 检索结果重排 (Rerank)

核心问题分析：在经过混合检索后，我们通常得到一批候选文档片段。然而其中有的片段可能只与查询部分相关，真正最相关的片段未必排在第一。直接将这些结果传递给生成模型，可能因噪声片段导致回答偏离。**重排 (Reranking)** 技术通过更精细的模型对候选结果重新排序，提升最终准确率 ([超越向量检索！混合检索 + 重排序改善 RAG 应用 | 新程序员-CSDN博客](#))。常用的重排模型是**Cross-Encoder**架构：将查询和候选段落拼接输入一个Transformer模型，直接输出一个相关性分数，然后根据分数对候选段落排序 ([Retrieve & Re-Rank — Sentence Transformers documentation](#))。由于Cross-Encoder在编码时考虑到了查询和文档之间的双向交互 (Attention)，相关性判断更准确，但计算成本较高，只适合对少量候选做精排 ([Retrieve & Re-Rank — Sentence Transformers documentation](#))。

案例场景：用户问题「最近公司的车险理赔流程是什么？」，混合检索返回了多个候选段落，包括：1) 旧的车险理赔流程，2) 新的车险理赔流程，3) 其他保险理赔的通用说明等。BM25可能因为关键词匹配，把旧流程排在前面；向量检索也可能返回通用说明。**但用户实际需要最新的车险理赔流程。通过重排模型对“查询+段落”进行相关度评分，包含“最新流程”的段落将获得更高分，从而在重排序后位居第一，保证生成模型优先利用最相关内容回答。**

优化策略与原理：

1. **候选集获取：**首先利用混合检索得到较宽松的候选列表，例如Top 50段落。
2. **重排序模型选择：**可以使用预训练的Cross-Encoder模型（如微软提供的MS MARCO上训练的cross-encoder）或**微调一个领域重排模型**。由于我们关注中文金融领域，理想情况下应在相应领域问答数据上微调一个BERT或RoBERTa模型，用于判断“(问题, 文档段落)”的相关性。开源方案包括 **Cohere Rerank API** 或 BGE作者提供的**bge-reranker**模型 ([超越向量检索！混合检索 + 重排序改善 RAG 应用 | 新程序员-CSDN博客](#))。
3. **分数计算：**对候选列表中的每个段落，让Cross-Encoder模型将“查询”和“段落”作为输入，输出一个相关性得分（一般可以归一化到0~1区间，1表示非常相关） ([Retrieve & Re-Rank — Sentence Transformers documentation](#))。这一步会针对每个对执行Transformer前向，计算成本高，因此候

选列表不要太大（通常几十到上百）。如果原始候选较多，可先用轻量级筛选模型或进一步限定如只取来自顶级文档的段落等。

4. **结果排序与截断**：按照重排模型打分，将候选段落从高到低排序，取Top K（如K=5）作为最终提供给生成回答的依据（[超越向量检索！混合检索 + 重排序改善 RAG 应用 | 新程序员-CSDN博客](#)）。由于LLM有输入长度限制，而且过多不相关段落会降低回答质量，限制Top K的大小很重要（[超越向量检索！混合检索 + 重排序改善 RAG 应用 | 新程序员-CSDN博客](#)）。经验上，即使上下文窗口足够大，也不宜直接给模型喂入大量低相关度内容。
5. **融合**：重排序可以应用于**混合检索的合并结果**（如本场景），也可以在**单一检索模式**后使用。例如仅用BM25检索也能在其结果后加一层语义重排来提升效果（[超越向量检索！混合检索 + 重排序改善 RAG 应用 | 新程序员-CSDN博客](#)）。因此，重排策略通用性强，能显著改善最终顶层结果的准确性。

实际效果：引入重排后，检索阶段的**准确率**明显提高——真正相关的文档片段更有可能排在首位。例如之前的例子中，重排模型会根据“最近”“车险”“理赔流程”等语义，将最新流程的段落提到第一位，即使BM25最初把旧流程排在上面。这样最终用于作答的知识是正确的**最新流程**信息，提升答案准确性。另外，对于那些查询中含模糊词的情况，重排模型能根据上下文判断相关性，过滤掉一些虽然匹配但不相关的片段，减少错误干扰。

示例代码：Cross-Encoder 重排候选结果

下面的代码示例展示如何使用Sentence-Transformers提供的CrossEncoder模型对候选文本进行重排。假设我们已经通过混合检索得到了若干候选文档片段 `candidates` 列表。我们选用一个公开的Cross-Encoder模型（例如MS MARCO数据上训练的MiniLM模型）对这些候选进行相关性评分，并输出按分数排序的结果。实际应用中，应替换为中文领域微调的模型或预训练中文Cross-Encoder模型。

```

1 from sentence_transformers import CrossEncoder
2 query = "最近公司的车险理赔流程是什么？"
3 candidates = [
4     "公司2020年的车险理赔流程包括报案、查勘定损、提交资料、理赔审核和赔付。",
5     "最新的车险理赔流程（2023年更新）为：在线报案->现场查勘->材料上传->理算审核->赔款支付。",
6     "车险理赔是指车辆发生保险事故后，被保险人向保险公司申请赔偿的过程。"
7 ]
8 初始化Cross-Encoder重排模型（此示例用英文MiniLM模型，占位）
9 reranker = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-6-v2') # 实际应使用中文微调模型
10 将查询和每个候选组成对，进行相关性预测
11 pair_inputs = [(query, doc) for doc in candidates]
12 scores = reranker.predict(pair_inputs)
13 将候选及得分配对，按分数排序
14 ranked_results = sorted(zip(candidates, scores), key=lambda x: x[1], reverse=True)
15 for doc, score in ranked_results:
16     print(f"Score: {score:.4f} | Passage: {doc[:30]}...")

```

说明：

- `CrossEncoder('cross-encoder/ms-marco-MiniLM-L-6-v2')` 加载一个跨编码器模型，此模型能对输入的(query, passage)对输出一个相关性分数。需要注意这个示例模型是英文的，仅作参考。实际应选择中文模型，例如`UER/roberta-base-finetuned-ranker`或自行微调的模型。
- `reranker.predict(pair_inputs)` 对每个(query, 文本)对进行评分。CrossEncoder内部会将query和文本拼接输入Transformer，输出一个logits分数表示相关性高低。由于我们只关心排序，相对分数即可。
- 根据得到的 `scores` 排序候选文本。打印结果可以看到各段落的相关性分数，从而验证最新流程的段落得分最高。

通过重排，我们确保了**最相关**的文档片段排列在前，提升了检索准确率，进而提高问答结果的准确性和用户满意度。

4.5：评估指标与效果评估

核心问题分析：为了验证以上优化是否提升了系统性能，我们需要引入科学的**评估指标**来量化召回率和准确率的改进。信息检索领域常用的指标有：

- **MRR (Mean Reciprocal Rank, 平均倒数排名)**：关注第一个相关结果出现的位置，反映用户是

否能很快找到答案 ([Evaluation Metrics for Search and Recommendation Systems | Weaviate](#))。MRR是所有查询 Reciprocal Rank 的平均值，其中每个查询的 Reciprocal Rank = $1/(\text{相关结果的排名})$ 。如果相关结果总是排在第一，MRR=1；如果相关结果平均排在第三位，MRR≈0.33。MRR 适合评估问答场景下**第一个正确答案**的易得性。

- **NDCG (Normalized Discounted Cumulative Gain, 归一化折损累计增益)**：考察整个排名列表的质量，包括多个相关结果的贡献 ([Evaluation Metrics for Search and Recommendation Systems | Weaviate](#))。它考虑结果的相关性等级和排名次序，通过折损因子（如 $1/\log_2(\text{rank}+1)$ ）给排名靠后的相关结果降低权重 ([Evaluation Metrics for Search and Recommendation Systems | Weaviate](#))。NDCG进行归一化以便不同查询间可比，值在0到1之间，1表示理想排序。NDCG@K通常用于评估Top K结果的综合相关性排序。
- **Precision@K (P@K, 前K精度)**：衡量在返回的前K个结果中，有多少比例是相关的 ([Evaluation Metrics for Search and Recommendation Systems | Weaviate](#))。例如Precision@5 = 前5个结果中相关结果数量/5。它直接反映用户看前K条结果能找到多少正确答案，不考虑顺序（非rank-aware指标）。常和Recall@K（在所有相关文档中前K找到多少）一起使用。
- （可选）**Recall@K (召回率)**：相关文档中有多大比例在前K结果里。由于问答系统往往每问只需一两个相关片段即可回答，有时Precision和MRR更受关注，但在多文档综合场景下Recall也重要。

通过这些指标，我们可以全面评估**召回能力**(是否找全相关内容)和**精确程度**(相关内容是否排在前面)。优化后的系统应在这些指标上优于基线系统。

案例场景：针对内部准备的一套测试查询（例如50个涵盖报销制度、保险销售技巧等问题），我们人工标注每个查询对应的相关文档片段作为**黄金标准**。然后对比：

- **基线系统**（仅向量检索）的结果：可能一些查询相关片段排在较后面，MRR较低（因为用户得翻很后才能找到答案）；Precision@3也可能偏低（前3结果中相关的不多）。
- **优化系统**（混合检索+微调+重排）的结果：预期MRR提高，相关片段更靠前；NDCG提升，表明整体排序更合理；Precision@3提高，前3就包含更多正确信息。

例如，查询「如何推销保险产品？」在基线系统中，也许第1相关片段排在第4位（RR=1/4=0.25），而优化后排到了第1位（RR=1）；多个相关片段都排在前列使NDCG@5接近理想值。这些改进都会反映在指标的提升上。

优化策略与原理：在优化过程中和上线前后，需持续评估指标：

- **确定评估集**：涵盖各种类型查询及对应的正确文档。对每条查询-文档对标注相关性等级（相关或不相关，或使用等级0/1/2表征完全无关、部分相关、非常相关）。
- **计算指标**：编写脚本计算每种指标。在Python中，可使用现有评测库如 `pytrec_eval` 来方便计算Precision、Recall、NDCG等 ([Evaluation Metrics for Search and Recommendation Systems | Weaviate](#)) ([Evaluation Metrics for Search and Recommendation Systems | Weaviate](#))。也可自行根据公式实现。

- **对比实验：**分别运行基线和各优化方案，记录指标值。例如，对比“仅向量” vs “混合检索” vs “混合+微调” vs “混合+微调+重排”，观察指标变化。
- **分析结果：**如果某些查询的指标未提升，深入分析原因（例如某查询意图分类错误导致没用对策略）。不断迭代调整。

通过量化指标，我们可以直观证明每个优化点的价值。例如混合检索可能主要提升**Recall@K**（召回率）而Cross-Encoder重排显著提升**MRR**和**NDCG**（排序质量）。在验证达到预期提升后，再将新方案应用到生产。

示例代码：计算 MRR、NDCG、P@K 指标

以下示例代码展示如何根据人工标注的相关文档列表和检索结果计算 MRR 和 P@K；同时给出NDCG的计算思路。假设我们有人为构造的小测试集，其中 `ground_truth` 列出每个查询相关的文档ID列表，`retrieved` 列出检索系统返回的文档ID按顺序排列。我们以此计算各指标。

```

1  @假设有两个查询的测试集
2  ground_truth = {
3      "Q1": [2, 5],      # 查询Q1的相关文档ID为2和5
4      "Q2": [1]         # 查询Q2的相关文档ID为1
5  }
6  retrieved = {
7      "Q1": [5, 2, 8, 3, 10], # 系统返回的文档顺序, 5号和2号是相关, 其它无关
8      "Q2": [3, 7, 1, 9, 4]   # 返回顺序中, 第3个结果ID=1是相关
9  }
10 def calculate_mrr(gt, res):
11     """计算平均倒数排名 (MRR) """
12     total_reciprocal_rank = 0.0
13     num_queries = len(gt)
14     for q, relevant_docs in gt.items():
15         rank = 0
16         for idx, doc_id in enumerate(res.get(q, []), start=1):
17             if doc_id in relevant_docs:
18                 rank = idx
19                 break
20         if rank != 0:
21             total_reciprocal_rank += 1.0 / rank
22     return total_reciprocal_rank / num_queries
23 def calculate_precision_at_k(gt, res, K=3):
24     """计算每个查询的Precision@K, 并返回平均Precision@K"""
25     precisions = []
26     for q, relevant_docs in gt.items():
27         retrieved_k = res.get(q, [])[:K]
28         if not retrieved_k:
29             continue
30         # 计算前K结果中相关文档的比例
31         rel_count = sum(1 for doc_id in retrieved_k if doc_id in relevant_
docs)
32         precisions.append(rel_count / K)
33         print(f"{q}的Precision@{K}: {rel_count}/{K} = {rel_count/K:.2f}")
34     # 返回平均Precision@K
35     return sum(precisions) / len(precisions) if precisions else 0.0
36 mrr_value = calculate_mrr(ground_truth, retrieved)
37 precision3 = calculate_precision_at_k(ground_truth, retrieved, K=3)
38 print(f"MRR = {mrr_value:.3f}")
39 print(f"平均 Precision@3 = {precision3:.3f}")
40 计算NDCG@K
41 def calculate_ndcg_at_k(gt, res, K=3):
42     """计算平均 NDCG@K (相关性二值相关0/1情况) """
43     total_ndcg = 0.0
44     num_q = len(gt)

```

```

45     for q, relevant_docs in gt.items():
46         # 计算DCG@K
47         dcg = 0.0
48         for idx, doc_id in enumerate(res.get(q, [])[:K], start=1):
49             rel = 1.0 if doc_id in relevant_docs else 0.0 # 二级相关性: 相
关=1, 否则=0
50             if idx == 1:
51                 dcg += rel
52             else:
53                 dcg += rel / math.log2(idx) # 折损: log2(rank)
54         # 计算IDCG@K (理想情况下的DCG)
55         # 将相关文档都假设排在最前面
56         ideal_rels = sorted([1.0]*len(relevant_docs) + [0.0]*K, reverse=True)[:K]
57         idcg = 0.0
58         for idx, rel in enumerate(ideal_rels, start=1):
59             if idx == 1:
60                 idcg += rel
61             else:
62                 idcg += rel / math.log2(idx)
63         ndcg = dcg / idcg if idcg > 0 else 0.0
64         total_ndcg += ndcg
65         print(f"{q}的NDCG@{K}: {ndcg:.3f}")
66     return total_ndcg / num_q
67 avg_ndcg3 = calculate_ndcg_at_k(ground_truth, retrieved, K=3)
68 print(f"平均 NDCG@3 = {avg_ndcg3:.3f}")

```

代码说明:

- `calculate_mrr` : 对每个查询, 在检索结果列表中找到第一个相关文档的位置, 取其倒数 (1/位置)。MRR是所有查询倒数排名的平均值。如果某查询没有任何相关文档被召回, 则记为0贡献。
- `calculate_precision_at_k` : 计算每个查询前K个结果的相关文档比例, 这里同时打印每个查询的Precision@K并计算平均值。
- `calculate_ndcg_at_k` : 演示NDCG@K的计算。对于每个查询, 计算DCG@K: 相关文档在第rank位贡献值 = $1/\log_2(\text{rank})$ (假设相关性等级为1, 非相关为0); 计算理想DCG (IDCG) 假设相关文档都排在最前面然后折损; $\text{NDCG} = \text{DCG}/\text{IDCG}$ 。代码中假定相关性非等级区分, 仅相关与否二值。如果有等级 (比如相关性打分0,1,2), 计算时 `rel` 可取相应等级值, IDCG则是将最高等级相关排序得到。

根据这个示例数据, 程序会输出每个查询的Precision@3和NDCG@3, 以及总体的平均值。例如可能输出:


```
1 Q1的Precision@3: 2/3 = 0.67
2 Q2的Precision@3: 1/3 = 0.33
3 MRR = 0.583
4 Q1的NDCG@3: 0.789
5 Q2的NDCG@3: 0.500
6 平均 NDCG@3 = 0.644
```

这些指标可以帮助我们判断优化是否有效。若应用混合检索和重排后，更多查询的相关文档出现在结果顶部，则MRR和Precision@3会提高；如果排序更理想，NDCG也会提升。通过持续监控MRR、NDCG、P@K等指标 (Evaluation Metrics for Search and Recommendation Systems | Weaviate) (Evaluation Metrics for Search and Recommendation Systems | Weaviate)并与基线对比，我们能量化证明系统在召回率和准确率上的改进。

4.6 召回总结

综合上述优化方案：

- **混合检索**：结合BM25和向量相似度，弥补单一检索缺陷，确保不同类型查询都能召回相关文档 ([超越向量检索！混合检索 + 重排序改善 RAG 应用 | 新程序员-CSDN博客](#))。
- **Embedding模型微调**：在金融保险语料上细调Embedding，使模型更懂领域语言，提高语义匹配效果 ([Fine-Tune Embedding: The Secret to Improve Response Rates | iWeaver AI](#))。
- **结果重排**：采用Cross-Encoder对候选段落重新打分排序，显著提升相关结果排名靠前的概率 ([超越向量检索！混合检索 + 重排序改善 RAG 应用 | 新程序员-CSDN博客](#))。
- **评估指标**：用MRR、NDCG、P@K等指标验证召回和排序效果的提升，指导迭代优化。

每个优化点相辅相成：混合检索和模型微调侧重**提高召回率**，重排序侧重**提升准确率**，最终使RAG问答系统能更准确地找到并提供答案。在实际落地时，应根据指标反馈不断调整策略，例如调整BM25与向量结果权重、增加训练数据微调Embedding、引入更强大的重排模型等，以持续优化用户提问的解答效果。通过完整的方案和持续评估，我们有信心显著改善金融保险场景下RAG问答系统的性能，提升用户满意度。

生成阶段优化

5.1 核心问题

1. **多轮对话的语义连贯性不足**：在用户多轮提问时，如果新问题是上一轮回答的跟进（例如用户问：“这个怎么申请？”），系统需要理解“这个”指代什么。缺少对话上下文的关联可能导致LLM误解提问，给出不相关或幻觉的答案。
2. **多模态知识的利用困难**：金融保险领域的知识库包含PDF手册、PPT演示、文本说明、视频讲解等多种形式。如果不对这些不同格式的数据进行预处理和结构化，检索时可能遗漏关键信息，导致答案不全面。
3. **缺少来源引用降低可解释性**：用户希望了解答案出处以建立信任。如果生成的答案没有标注来源，用户无法追溯信息真实性。特别是从长文档提取内容时，不注明具体出处会降低答案的可信度和可检查性。

5.2 实际案例（Case）

假设一位用户向保险问答系统咨询：

用户：我有一款寿险产品ABC，它的保障范围是什么？ **系统**：根据我们的资料，ABC寿险涵盖身故、全残保障，并提供重大疾病附加险【来源：寿险ABC产品手册 第10页】。

接着用户继续问：

用户：这个怎么申请？（这里“这个”指代寿险产品ABC）

如果没有上下文连贯性，系统可能无法理解“这个”指什么，导致回答偏离主题或出错。例如，系统可能回答与“申请”无关的内容，或者胡乱生成申请流程。相反，经过优化的系统会识别这是上一问题的跟进，自动将“寿险产品ABC”作为上下文加入检索，从而找到PPT文件或PDF手册中关于申请流程的内容。然后系统回答：

系统：您可以通过以下步骤申请ABC寿险：【来源：ABC寿险申请流程PPT 第2页】首先，填写申请表并准备身份证明；然后，将材料提交至客服中心或通过官方网站上传；最后，等待审核通知。该流程平均需要5个工作日【来源：客服视频讲解 00:02:10】。

这个案例展示了：如果不考虑对话上下文，系统会在第二问出现错误；而结合多模态知识（PPT中的申请流程、客服视频中的时长信息）并标注来源，能提供连贯且可信的回答。

优化策略

为解决上述问题，我们可以从以下几个方面优化生成阶段：

- **维护对话上下文，确保连贯**：在每次用户提问时，检测问题是否包含代词或省略（如“这个”“它”等）以判断是否为跟进问答。如果是跟进问题，将之前的相关问答摘要或关键术语添加到当前查询中。一种常见做法是**查询重写**（Query Rewriting）：将用户的新问题与上下文合并重写成完

整问题，再送入检索和LLM。与此同时，系统应维护一个对话历史状态，让LLM参考之前的问答或已检索的知识，避免因缺少背景造成误解。

- **整合多模态知识，提高答案全面性：**预先对PDF、PPT、文本、视频等资料进行解析和结构化处理，存入统一的向量数据库以便检索。比如：
 - **PDF/PPT：**提取文字内容，保留章节标题、表格数据等结构信息，将长文档按段落或页面切分成知识片段（chunks），并为每个片段添加文档名称、页码/幻灯片编号等元数据。
 - **视频：**对讲解视频执行语音转文本（ASR），获得字幕稿。根据时间戳将字幕稿切分成短段，并存储视频ID和时间段元数据。必要时可结合视频说明文字或关键帧截图的文字说明。
 - 将不同模态的数据转换为统一的文本嵌入向量，以使用同一种检索方式获取相关片段（例如使用同一嵌入模型表示文本和语音转文本）。或者采用**多模态检索策略**：分别在文本库、图像/视频库中检索，再融合结果。
 - 在生成答案时，允许LLM综合多个来源的片段。例如同时引用保单PDF中的条款和培训视频中的说明，以形成完整答案。
- **在答案中加入来源引用，增强可解释性：**设计提示（Prompt）要求LLM在给出答案时**标注信息来源**。例如，让模型在句末用括号注明来源文档名称或索引编号。实现方法可以是：在将检索到的文档片段传递给LLM时，附加标记（如【1】、【2】）或者直接提供“引用格式”的文本，让模型仿照引用格式回答。对于长文档的引用，如果答案来自同一资料的不同部分，可以拆分引用为【文档A，第10页】、【文档A，第15页】等，精确指明出处。生成策略上，可以使用RAG的**引用增强模式**，即模型严禁脱离提供的知识片段编造答案，确保每句都有据可依。最终答案输出时，将源文件名称或链接映射为用户可查看的引用，以使用户点开核实内容。这种动态插入引用的方式保证了答案的可溯源性，减少幻觉，增加用户信任。

代码示例（GPT-4 适用）

下面提供一个简化的Python伪代码示例，演示多轮RAG问答系统如何融合对话上下文、多模态检索和来源引用标注（需结合实际向量数据库和LLM接口实现）：

```

1  初始化向量数据库，已预先加载PDF文本片段、PPT文本片段、视频字幕片段等多模态知识
2  vector_store = VectorStore(data=load_multimodal_chunks())
3  conversation_history = [] # 用于保存多轮对话的问答历史
4  def is_follow_up(question):
5      """简单判定问题是否为跟进问答（根据代词/省略等）"""
6      follow_keywords = ["这个", "那种", "这样", "怎么", "如何", "吗?"] # 简化
      判断逻辑
7      return any(kw in question for kw in follow_keywords)
8  def answer_question(user_question):
9      # 如果是跟进问题且有历史，则将上一次用户提问或主题融入当前问题
10     if conversation_history and is_follow_up(user_question):
11         last_topic = conversation_history[-1]["topic"] # 上一轮对话主题
12         # 将用户问题重写，加入主题关键字
13         rewritten_q = f"{last_topic}, {user_question}"
14     else:
15         rewritten_q = user_question
16
17     # 检索相关知识片段（返回内容和来源标识）
18     docs = vector_store.search(rewritten_q, top_k=3)
19     context = "".join([f"[{i}] {doc.content}\n" for i, doc in enumerate(docs, start=1)])
20
21     # 构造提示，要求GPT-4根据检索片段回答，并标注引用编号
22     prompt = (
23         "请根据以下资料回答用户问题，并在对应内容后标注来源编号：\n"
24         f"{context}\n"
25         f"用户问题：{user_question}\n"
26         "回答： "
27     )
28     # 调用GPT-4模型生成答案
29     answer = openai.ChatCompletion.create(model="gpt-4", messages=[{"role": "user", "content": prompt}])
30     answer_text = answer["choices"][0]["message"]["content"]
31
32     # 记录本轮主题（可选：从user_question或检索结果中抽取主题关键词）
33     topic = extract_topic(user_question, docs)
34     conversation_history.append({"question": user_question, "answer": answer_text, "topic": topic})
35
36     return answer_text
37 模拟对话：
38  q1 = "ABC寿险的保障范围是什么？"
39  print(answer_question(q1)) # 系统基于知识库回答，答案包含来源标注，例如[1]
40  q2 = "这个怎么申请？" # 跟进问答，指代上文提到的“ABC寿险”
41

```

```
print(answer_question(q2)) # 系统利用对话历史，将ABC寿险作为上下文，检索申请流程
```

上述代码示例演示了：当检测到跟进问题时，将上一轮主题合并到当前问题中用于检索；从多模态向量数据库中检索相关内容；在提示中加入来源标记并让GPT-4按照指定格式作答。实际应用中可以更复杂的对话状态管理和Prompt设计，但核心思想是一致的。

进阶优化方案（下一期）